DOCUMENT 13

Operations Manual

Installation, CLI, database, deployment, and incident response.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

System Requirements

Requirement	Minimum	Recommended	Notes
Python	3.11	3.12	FastAPI backend
Node.js	20 LTS	22 LTS	Frontend build
RAM	2 GB	4 GB	VLM peaks ~1.5 GB
Storage	5 GB	20 GB	Images + DB
Network	Outbound HTTPS	Low-latency	VLM API calls
Database	SQLite 3.x	PostgreSQL 15+	SQLite = dev only

Installation

bash — clone repository

```
$ git clone https://github.com/your-org/ngw-core.git
$ cd ngw-core
```

bash — Python environment setup

```
$ python3 -m venv .venv
$ source .venv/bin/activate          # macOS / Linux
$ .venv\Scripts\activate             # Windows
$ pip install --upgrade pip
$ pip install -r requirements.txt
```

bash — Frontend setup

```
$ cd ui
$ npm install
$ cd ..
```

First-Time Setup

Copy the environment template and fill in required values before starting the server.

.env.local — minimum required variables

```
# AI Gateway (choose one auth method)
AI_GATEWAY_API_KEY=gw_xxxxxxxxxxxxxxx
# OR use OIDC (preferred on Vercel):
# VERCEL_OIDC_TOKEN=<auto-provisioned by vercel env pull>

# Database
DATABASE_URL=sqlite:///./data/ngw_users.db

# CORS
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173

# Lab access (comma-separated emails)
LAB_ALLOWED_EMAILS=you@org.com,teammate@org.com

# Cron authentication
CRON_SECRET=your-random-secret-here
```

bash — initialize database and seed data

```
# Create tables
$ python3 -c "from db.database import init_db; init_db()"

# Load starter dataset (seeded signals + gold sets)
$ python3 scripts/seed_starter_dataset.py

# Verify dataset integrity
$ python3 scripts/verify_dataset.py
```

Development Server

Run backend and frontend in two separate terminal sessions.

terminal 1 — start backend (auto-reload on save)

```
$ source .venv/bin/activate
$ uvicorn main:app --reload --host 0.0.0.0 --port 8000
# INFO:     Uvicorn running on http://0.0.0.0:8000
# INFO:     Application startup complete.
```

terminal 2 — start frontend (Vite HMR)

```
$ cd ui
$ npm run dev
# VITE v5.x  ready in 300 ms
# ➜  Local:   http://localhost:5173/
# ➜  API:     http://localhost:8000/ (proxied)
```

QUICK VERIFY

- Open http://localhost:5173 — the home screen should load.
- Open http://localhost:8000/health — should return {"status":"ok"}.
- Open http://localhost:8000/docs — FastAPI auto-generated Swagger UI.

Database Operations

bash — run schema migrations

```
$ python3 scripts/run_migrations.py
# Migrations are idempotent — safe to re-run.
# Always backup before migrating in production.
```

bash — backup and restore (SQLite)

```
# Backup
$ sqlite3 data/ngw_users.db ".backup data/backup_$(date +%Y%m%d).db"

# Verify backup
$ sqlite3 data/backup_$(date +%Y%m%d).db ".tables"

# Restore
$ cp data/backup_YYYYMMDD.db data/ngw_users.db
```

bash — inspect schema and table sizes

```
$ sqlite3 data/ngw_users.db
sqlite> .tables
sqlite> .schema session_signals
sqlite> SELECT COUNT(*) FROM session_signals;
sqlite> SELECT signal_source, COUNT(*) FROM session_signals GROUP BY 1;
sqlite> .quit
```

Environment Variables — Complete Reference

Variable	Type	Required	Default	Description
AI_GATEWAY_API_KEY	string	alt.	—	Vercel AI Gateway key (or use OIDC)
VERCEL_OIDC_TOKEN	string	alt.	auto on Vercel	Short-lived JWT; preferred in prod
AI_MODEL_STRING	string	No	gateway default	VLM model identifier for AI Gateway
DATABASE_URL	string	Yes	sqlite:///./data/…	SQLite path or Postgres DSN
ALLOWED_ORIGINS	string	Yes	localhost:3000,5173	Comma-separated CORS origins
UPLOAD_DIR	string	No	static/uploads	Directory for uploaded images
LAB_ALLOWED_EMAILS	string	Yes	—	Comma-separated Lab-access emails
CRON_SECRET	string	Yes	—	Auth header for scheduled jobs
LOG_LEVEL	string	No	INFO	DEBUG for verbose output
MAX_UPLOAD_MB	integer	No	10	Max image upload size in MB
VLM_TIMEOUT_SECONDS	integer	No	30	Per-pass VLM call timeout
VLM_PASS_COUNT	integer	No	3	Analysis passes per image

CONFIDENCE_EARLY_EXIT	float	No	0.94	Skip remaining passes if exceeded

Running Tests & Benchmarks

bash — test suite

```
# Full test suite
$ python3 -m pytest tests/ -q --tb=short

# Specific test files
$ python3 -m pytest tests/test_shoot_match.py -v
$ python3 -m pytest tests/test_engine.py -v
$ python3 -m pytest tests/test_signals.py -v

# Stop on first failure
$ python3 -m pytest tests/ -x --tb=long
```

bash — benchmarks and validation

```
# Pattern matching benchmarks
$ python3 scripts/run_benchmarks.py

# CI benchmark (used in CI/CD pipeline)
$ bash scripts/ci_benchmark.sh

# Validate system YAML configurations
$ python3 scripts/validate_system_yamls.py

# Validate reference catalog and packs
$ python3 scripts/validate_catalog_and_packs.py
```

Production Deployment — Vercel

bash — Vercel setup and deploy

```bash
# Install Vercel CLI
$ npm i -g vercel

# Link to Vercel project (first time only)
$ vercel link

# Pull environment variables (provisions OIDC token)
$ vercel env pull .env.local

# Deploy to preview
$ vercel deploy

# Deploy to production
$ vercel deploy --prod

# Inspect deployment
$ vercel inspect <deployment-url>
```

vercel.json — cron jobs configuration

```json
{
  "crons": [
    {
      "path": "/api/cron/nightly-benchmark",
      "schedule": "0 3 * * *"
    },
    {
      "path": "/api/cron/learning-sweep",
      "schedule": "0 4 * * 1"
    }
  ]
}
```

Health Monitoring

bash — health check commands

```
# Service liveness
$ curl http://localhost:8000/health
# → {"status":"ok","version":"1.0","uptime_s":3621}

# Database status
$ curl http://localhost:8000/health/db
# → {"tables":5,"schema_version":"2025-03","ok":true}

# Signal hygiene summary (Lab auth required)
$ curl -H "Authorization: Bearer $AI_GATEWAY_API_KEY" \
       -H "x-lab-email: you@org.com" \
       http://localhost:8000/api/lab/signals/summary
# → {"live":1482,"seeded":500,"internal":23,"expert_review":8,
#    "learning_eligible":1482,"metrics_eligible":1482}
```

bash — log tailing and analysis

```
# Tail logs in development (uvicorn output)
$ uvicorn main:app --reload 2>&1 | tee -a logs/dev.log

# Filter for errors
$ grep "ERROR\|Exception\|500" logs/dev.log

# Count VLM timeouts in the last 24 hours
$ grep "VLM_TIMEOUT" logs/dev.log | grep "$(date +%Y-%m-%d)" | wc -l

# Watch for Lab access attempts
$ tail -f logs/dev.log | grep "lab-access"
```

Maintenance Procedures

Weekly

- Review open candidates — approve or reject items older than 7 days.
- Check gold set evaluation scores — flag any below 80%.
- Review signal hygiene summary — confirm live count is growing.
- Check VLM timeout rate — should stay below 2% of requests.

Monthly

- Run full gold set re-evaluation after any engine update.
- Rotate AI Gateway API key or verify OIDC refresh is working.
- Audit LAB_ALLOWED_EMAILS — remove any leavers.
- Archive seeded signals older than 90 days (they skew storage).
- Run python3 scripts/nightly_benchmark.py manually to verify regression baseline.

Incident Response Playbook

API RETURNS 500 ON /API/SHOOT-MATCH

- 1. curl http://localhost:8000/health/db confirm DB is accessible.
- 2. Check LOG_LEVEL=DEBUG output for exception traceback.
- 3. Confirm AI_GATEWAY_API_KEY / OIDC token is valid and not expired.
- 4. curl the /api/lab/analyze endpoint with the same image — compare responses.

VLM CALLS FAILING / TIMING OUT

- 1. Test gateway connectivity: curl https://api.vercel.ai/health
- 2. Check VLM_TIMEOUT_SECONDS — increase to 60 if inference is slow.
- 3. Check AI Gateway quota in the Vercel Dashboard.
- 4. Fallback: set AI_MODEL_STRING to a lighter model temporarily.

GOLD SET SCORES DROPPED AFTER ENGINE UPDATE

- 1. Identify which candidate was recently applied (check candidates table).
- 2. Review the candidate proposed_change JSON for the problematic change.
- 3. Revert: UPDATE candidates SET status='proposed' WHERE id='...';
- 4. Re-run gold set evaluation to confirm scores recover.
- 5. File a new candidate with more conservative proposed_change values.